How cost-efficient is scaling up versus scaling out Valkey for different read/write workloads?

Eryk Kściuczyk
Technische Universität Berlin
Berlin, Germany
e.ksciuczyk@campus.tu-berlin.de

***Abstract*—This paper benchmarks the cost efficiency of scaling Valkey vertically and horizontally under changing read/write workloads on Google Cloud Platform. We compare single-node and clustered deployments across four price ranges using throughput, latency percentiles, and requests-per-dollar as the main metrics. The results show that single-node deployments are most cost-efficient at low budgets, while horizontal scaling becomes advantageous in the mid-range, where it delivers the highest overall performance and better efficiency than a comparable larger single VM. At the high-performance tier, both strategies show diminishing returns, indicating practical scaling limits in the tested configurations. Overall, the findings suggest scaling up for budget and entry-level deployments, scaling out in the mid-range, and avoiding overprovisioning beyond that point.**

***Index Terms*—benchmark, cloud, Valkey, vertical scalability, horizontal scalability, cost**

I. Introduction

There are many use cases for in-memory key-value databases in modern development. They can be used as a cache, for session management, message queues, storing the state of online games, and much more[1]. Company and individual users should concern themselves with the cost of operating a system or a service. Redis, a highly popular in-memory database, changed its licensing model on March 20, 2024[2]. This raised concerns in the community and among cloud providers about long-term openness, redistribution, and the future cost of Redis-compatible managed services. Valkey emerged as a BSD-licensed, community-governed fork intended to preserve an open Redis-compatible ecosystem[3]. Managed-service adoption also accelerated because major cloud providers began integrating Valkey support into their caching products, in some cases positioning it as a lower-cost alternative to Redis-compatible offerings[4]. Valkey also shows higher performance than Redis [1]. Additionally, it is fully Redis-compatible, making the transition smooth for developers familiar with Redis. As the demand for any service can grow, the development team should ask themselves during development whether their solution is scalable, especially when they plan to support many users concurrently. Valkey supports two scaling methods, scaling out using a cluster mode, and scaling up using IO-threads. There are some benchmarks from other users suggesting the scalability limits of IO-threads, as the performance still mostly relies on main-thread execution[5], yet there are no scientific papers concerning Valkey's scalability at the time of writing.

Therefore we want to fill this research gap by benchmarking not only the vertical and horizontal scalability of Valkey, but also the cost efficiency of both strategies.

The target system we designed this experiment for uses Valkey as a cache service. Such a system needs to perform well in a few qualities, but we will focus on the two primary ones for any caching service:

- have high throughput
- have low latency

We run the same benchmarks on multiple setups with different numbers of VMs, cluster sizes, and CPU cores.

In particular, we will answer the following questions in this paper:

- How cost-efficient is scaling up versus scaling out Valkey for different read/write workloads?
- Is there a scaling limit to the IO-threads option in Valkey for vertically scaled setups?
- Is there a scaling limit to horizontally scaled Valkey?

II. Background

Cloud service benchmarking evaluates a system from the perspective of a client interacting with a deployed service. Prior work stresses that such benchmarks should define clear quality goals, representative workloads, and reproducible measurement procedures [2]. In our case, the most relevant qualities are throughput, latency, and cost efficiency.

Throughput measures how many requests per second a system can sustain. For an in-memory cache, high throughput is important because the service is typically placed on the critical path of many application requests. Latency measures the response time perceived by the client. Since average latency can hide rare but important slow requests, percentile-based metrics such as p50, p99, and p99.9 are commonly used to capture both typical and tail behavior [3].

Cost efficiency relates performance to the monetary cost of the infrastructure. In cloud environments, raw throughput alone is often insufficient for decision-making, because a faster configuration may still be less attractive if its price grows disproportionately. For this reason, benchmarking cloud

[1]https://valkey.io/
[2]https://redis.io/blog/redis-adopts-dual-source-available-licensing/
[3]https://valkey.io/
[4]https://aws.amazon.com/elasticache/
[5]https://github.com/valkey-io/valkey/issues/2022

systems should consider not only absolute performance but also the relation between performance and cost [2], [4].

A further challenge is cloud environment variability. Unlike dedicated hardware, public cloud resources may exhibit performance fluctuations caused by factors such as shared infrastructure, noisy neighbors, and time-dependent contention. As a result, repeated measurements are necessary in order to distinguish stable trends from incidental variation [2], [5].

A. System Under Test (SUT)

III. Benchmark Design

A. Benchmark Objectives

The benchmark goals are to measure throughput, latency, and success rate in multiple hardware configurations. We chose those metrics because they are relevant to database performance [2], [4]. We chose success rate in order to measure whether the increased amount of RAM will lead to more cache hits under semi-realistic load.

We want to compare which of two scaling strategies, horizontal or vertical, is more efficient. In other words, how much additional throughput, latency improvement, and success rate does a dollar give us.

B. Use Case and Workload

We generate load on a system by simulating semi-realistic cache database traffic using our custom data generator. The traffic simulates only two standardized Redis requests, SET and GET. The operations are performed on keys sampled from a Zipf distribution (exponent:1.1, offset:1.0), as this resembles realistic traffic, where some keys are retrieved much more often than others [6]. We perform a micro-benchmark as all the key operations are independent.

We use a seeded random data generator so that we have consistency between runs, thus ensuring reproducibility and helping with repeatability. The generated keys are of length 24, 7 characters for a prefix and 16 for the key. We use 7 characters for a prefix, as a common use case of Redis-compatible databases is to add a table name in the prefix, e.g., "users", and we chose 16 for the key as it is the length of a UUID, which is commonly used. This way we make the results of the benchmark more realistic.

The workload of the benchmark is dynamic and it starts with 10% GET requests and 90% SET and linearly shifts to 90% GET and 10% SET through the benchmark run.

We generate the load using a separate `t2d-standard-48` VM running in GCP[6] in the same region and zone as the SUT using our custom written tool. We use the same size of the load-generator VM for all the benchmark runs and configurations. During all runs we have monitored the resource utilization of the load-generator to ensure it is not the bottleneck. The load-generator uses a fixed pool of 256 workers, each having its own connection to the DB and sending another request as soon as it receives a response from the previous one until the benchmark finishes.

C. System Architecture

The architecture of our benchmark is straightforward, we have a load-generator node and an SUT node or nodes (depending on the scalability strategy). The load-generator sends requests and saves the results to a local CSV file, which is retrieved after a benchmark run and processed offline. After each benchmark run the whole infrastructure is removed, to ensure that one run does not influence another by keeping some data in cache.

The requests are sent directly to the appropriate SUT nodes as the client libraries used are cluster aware, thus they cache the key ranges that belong to each node, removing the need for a layer 7 load-balancer.

To ensure reproducibility we use infrastructure as code with `terraform` and bash scripts for setup. This way others may reproduce our benchmark by using the same machine types, network configuration, and setup and configuration of Valkey.

We use Google Cloud Platform (GCP) due to free credits provided by the university as benchmarking large VM sizes and clusters quickly gets expensive. We run the experiments in the `us-west1` region in zone `c`, due to the high default total core count quota for the `t2d` CPU type we chose. For the load-generator we use `t2d-standard-48` to match the largest benchmarked VM type for vertical scalability and the total core count of the largest horizontally scaled cluster.

All the nodes were deployed on the same cloud provider in the same region and zone to reduce network communication costs and potential variability in the connection.

In the table Table I we show the different VM setups along with their prices and the category they compete in. We defined four price categories for setups. Each category represents a price range:

1) Budget-entry-level ~14$/month
2) Entry-level ~50$/month
3) Mid-range ~200$/month
4) High-performance ~666$/month

The Budget-entry-level price range contains only a single-node setup as in this price range there were no viable options to form a cluster, as it was the cheapest VM of `t2d-standard` available. We included this category to see what performance one can achieve on a limited budget, as Valkey performance is still highly single-threaded.

[6]https://cloud.google.com/compute/docs/regions-zones

TABLE I
TABLE SHOWING ALL CATEGORIES AND THE VM SETUPS CHOSEN FOR THEM

VM Type	CPU Cores	Memory	Price
Budget-entry-level			
1x t2d-standard-1	1	4GB	$13.88
Entry-level			
1x t2d-standard-4	4	16GB	$55.52
3x t2d-standard-1	3x1	12GB	$41.64
Mid-range			
1x t2d-standard-16	16	64GB	$222.06
7x t2d-standard-2	7x2	56GB	$194.32
High-performance			
1x t2d-standard-48	48	192GB	$666.18
24x t2d-standard-2	24x2	192GB	$666.24
48x t2d-standard-1	48x1	192GB	$666.24

D. Benchmark Implementation

We use a custom benchmark tool implemented in Go by us, available on GitHub[7]. Go was chosen due to its strong performance as a compiled language and its great support for concurrency, making the code easier to understand for potential reviewers and increasing development velocity.

To improve repeatability we used `mise` to control the versions of tools alongside the standard dependency tracking of Go. By tools we mean gcloud, google-cloud-pricing-cost-calculator, Go, Terraform, and uv.

Each hardware configuration has been benchmarked 3 times to try to mitigate cloud provider performance variability. We run each benchmark for 20 minutes, the duration is limited but was chosen due to limited available GCP credits.

We don't give Valkey a warm-up time and start measuring the results from the beginning, the same applies to the shutdown time. To prevent unreliable data, we discard the first 2 minutes of the benchmark and the last 30 seconds of the benchmark run in offline analysis. The seconds at the end are discarded as some Goroutines may already start quitting and thus make the results not representative.

E. Experiment Variables

a) *Independent Variables:*

b) *Dependent Variables:*

We measured latency by saving the calculated time difference between sending a request and receiving a response. Additionally, we save the time when the request was sent and the request type (GET or SET). Afterwards, using this saved data, we calculate throughput.

F. Measurement Methodology

Data has been collected between 30.01.2026 and 04.02.2026. We collected the CSV result files from the load-generator using rsync. We observed CPU, memory, network, and disk utilization manually using `btop` during benchmark runs.

[7]https://github.com/erykksc/valkey-benchmark

IV. RESULTS

In this section we present our results by showing one table per price range. This way we compare the horizontally scaled setups with their price-equivalent vertically scaled setups.

A. Summary

First we show Table II with a summary of the results, and then we delve deeper into individual metrics.

TABLE II
SYSTEM PERFORMANCE AND COST EFFICIENCY ACROSS DEFINED PRICE CATEGORIES.

Type	Node Count	Throughput (RPS)	Latency p50 (ms)	Latency p99 (ms)	Latency p99.9 (ms)	Efficiency (RPS/$)
Budget Entry Level						
Single	1x 1cores	46488 ± 1671	4.01 ± 0.27	15.51 ± 0.87	21.68 ± 2.04	1507.41
Entry Level						
Single	1x 4cores	114801 ± 4993	2.14 ± 0.11	3.86 ± 0.15	9.58 ± 0.48	930.63
Cluster	3x 1cores	105270 ± 7063	1.08 ± 0.04	10.46 ± 1.45	15.11 ± 1.22	1137.82
Mid Range						
Single	1x 16cores	202438 ± 13911	1.19 ± 0.07	2.43 ± 0.37	6.09 ± 0.34	410.26
Cluster	7x 2cores	241606 ± 10513	1.04 ± 0.04	2.08 ± 0.18	14.66 ± 14.21	559.59
High Performance						
Single	1x 48cores	192225 ± 21297	1.22 ± 0.11	3.76 ± 0.64	7.91 ± 0.85	129.86
Cluster	24x 2cores	200404 ± 22836	1.24 ± 0.10	2.54 ± 0.90	9.61 ± 9.64	135.38
Cluster	48x 1cores	177921 ± 4921	1.41 ± 0.04	2.41 ± 0.05	4.71 ± 0.04	120.19

In Table II we see that in the Entry-level category, the single VM setup achieves higher throughput and lower p99 and p99.9 latencies with lower standard deviation compared to the cluster setup. On the other hand, the cluster setup achieves almost two times lower p50 latency with better price efficiency.

The situation changes in the Mid-range category, where the cluster achieves better throughput with smaller standard deviation, lower latencies in p50 and p99, and better efficiency. It is worth mentioning that we observed a high standard deviation of 14.21ms in the p99.9 latency in the cluster setup, which is 41.79 times higher than that of the single-node setup. Additionally there is an almost fivefold improvement in the p99 latency of the clustered setup in the Mid-range category compared to the Entry-level clustered setup.

In the High-performance category, we observe that we have reached the scalability limits of both strategies. Both the single-node and clustered setups performed worse than their respective setups in the Mid-range category. Furthermore, in this category we benchmarked two cluster setups, one with 24 VMs with 2 cores and one with 48 VMs with 1 core

each, where the setup with the lower number of VMs achieved higher throughput but with high variability of 22836 RPS. The setup of 48 VMs with 1 core showed lower p99 and p99.9 latencies, but with lower throughput than the 7 VMs with 2 cores from the Mid-range category.

B. *Cost Efficiency*

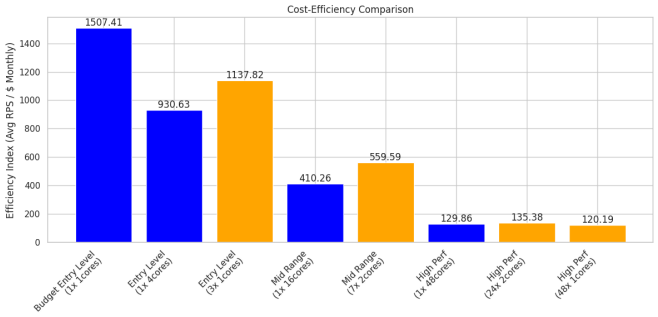

Fig. 1. Cost Efficiency Comparison plot for all VM setups benchmarked

In the Fig. 1 plot we can observe a trend that the vertically scaled setups achieve higher efficiencies than their comparable horizontally scaled ones.

Additionally we observe high cost efficiency for the Budget-entry-level setup, showing the high performance of a single-CPU single-VM setup.

Another trend in the results is that the more resources we use the lower the efficiency, both for vertically and horizontally scaled systems.

C. *Latencies*

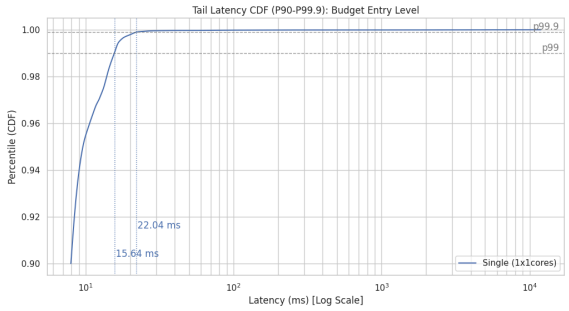

Fig. 2. Cumulative Distribution Function (CDF) of tail latencies in Budget-entry-level category

The Fig. 2 plot shows the cumulative distribution function (CDF) for the Budget-entry-level category where we see a smooth, almost vertical line reaching p99.9 at 22.04ms. We read the exact value from Table II.

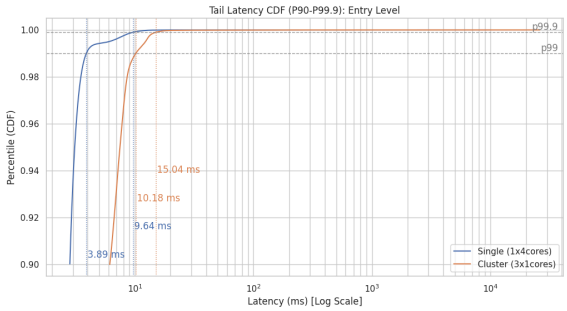

Fig. 3. Cumulative Distribution Function (CDF) of tail latencies in Entry level category

We compare the Entry-level setups in Fig. 3 and see that the tail latency CDF for the single-VM setup is steeper than the clustered one and reaches both p99 and p99.9 faster.

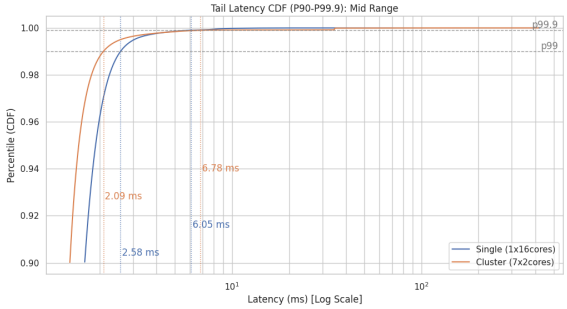

Fig. 4. Cumulative Distribution Function (CDF) of tail latencies in Mid-range category

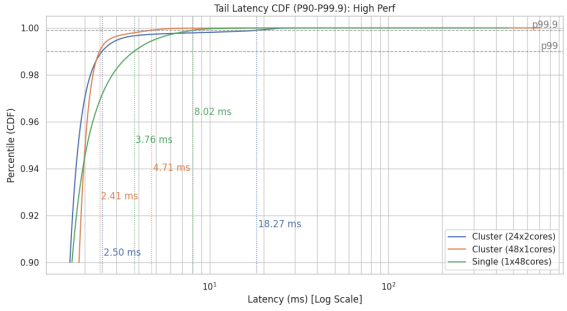

Fig. 5. Cumulative Distribution Function (CDF) of tail latencies in High-performance category

D. *Workload Transition*

We run the benchmark with a changing workload over time. The requests initially have a 90% probability of being SET requests and 10% GET requests and this shifts linearly with time to 10% SET and 90% GET requests. Thus we plot the throughput and p99 latency over time to see whether there are some patterns over time.

In Fig. 6 we see the workload transition plot for the Budget-entry-level category, where the throughput remains relatively stable despite the changing request mix.

Fig. 6. Workload Transition Plot portraying throughput (RPS) and p99 latency (ms) for Budget-entry-level

In Fig. 7 we see the workload transition plot for the Entry-level category, where the two setups differ clearly in both throughput stability and latency behavior.

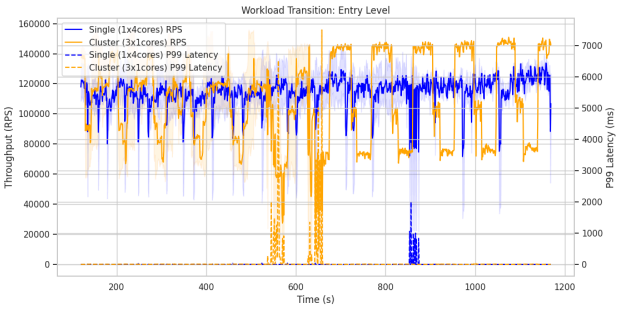

Fig. 7. Workload transition over time for the Entry-level setup, comparing single-node (blue) and cluster (orange) throughput and p99 latency. Solid lines show throughput, dashed lines show p99 latency, and the semi-transparent shaded regions in the background indicate variability in the measurements over time, highlighting the spread around the observed performance levels between benchmark runs.

In Fig. 7 we see that the cluster achieves higher peak throughput in the Entry-level category but suffers from significant instability. The single node delivers lower but much more stable throughput.

Both the cluster and single-VM setups contain periodic dips. In the second half of the experiment, where there are more GET requests, the clustered setup exhibits a clear periodic pattern where throughput alternates between two stable states: a prolonged high-throughput plateau and a prolonged low-throughput plateau. Transitions between these states occur swiftly, producing a square-wave–like oscillation in performance.

Additionally there are two clusters of latency spikes for the clustered setup and one such cluster for the single VM setup. The latency has almost zero variability outside of those clusters.

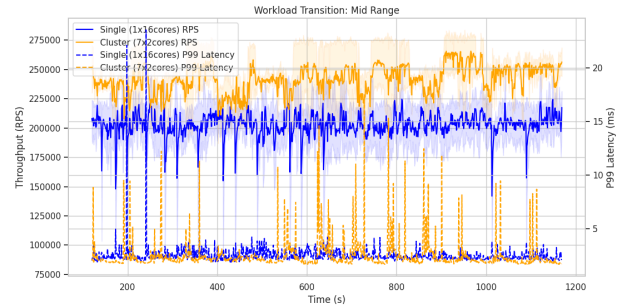

Fig. 8. Workload Transition Plot portraying throughput (RPS) and p99 latency (ms) for Mid-range

In Fig. 8 we see that the clustered setup achieved higher throughput, while the single-node setup appears more stable with occasional dips. The latency plot for the single-node setup is more consistent throughout the benchmark with periodic small latency spikes, whereas the clustered setup has lower latency but more sporadic, higher-magnitude latency spikes compared to the single-VM setup. The latency spikes in the clustered setup appear mostly on the right side of the plot, that is, during the time when there are more GET requests than SET requests.

Fig. 9. Workload Transition Plot portraying throughput (RPS) and p99 latency (ms) for High-performance

In Fig. 9 we compare the High-performance setups over time and we see more stable throughput in the clustered setups than in the single-VM setup. The throughput in the single-VM setup shows steep dips in performance.

The latencies in the two clustered setups include periodic spikes, while the single VM setup has lower periodic spikes with occasional very high ones.

V. Evaluation/Discussion

The length of the benchmark runs in this experiment was relatively short, and it would be preferable to run each of them for longer, e.g., 1 hour instead of 20 minutes. This way one could obtain more reliable results, and observe the trends better.

The workload in the experiment is semi-realistic as it uses a Zipf distribution but only simple commands, SET and GET, benchmarking only the basic functionality of Valkey. Redis-compatible databases such as Valkey provide a wide set of

commands whose performance remains to be benchmarked and could be explored in future research.

The cloud environment introduces variability into the measurements, as performance may vary between different times of the day due to a noisy-neighbor problem and on a day-to-day basis. We likely observed this variability in the p99.9 latency measurements of the Mid-range setup. It would be preferable to rerun those benchmark runs, but we already exceeded the assigned credits and our time was limited.

VI. Conclusion

The results show clear trends where the single-VM deployments achieve higher throughput, lower latencies, and higher efficiency compared to the clustered VM setups. Additionally, single-node deployments are easier to maintain and deploy. Thus it is suggested to use a single-VM setup for the Budget-entry-level and Entry-level categories.

For the Mid-range category we see that the horizontally scaled system outperforms the single-VM deployment in all performance metrics as well as in efficiency. The only downside seems to be the p99.9 performance, but our results show very high variability, making the result unclear. The variance could come from the relatively low benchmarking time, 1 hour combined (3x 20-minute runs), and cloud service performance variability. This interpretation is supported by the fact that the High-performance Cluster setups had lower p99.9 latency, while being worse in all other metrics. Thus, we cannot say that the high p99.9 variability in the clustered Mid-range setup is inherent to the clustered deployment, and it is more likely due to other factors.

In the High-performance category we see that the setups perform worse than the Mid-range setups, indicating that some scaling bottleneck has been hit. In the vertically scaled setup, this is likely due to how Valkey utilizes additional threads, using them only for IO, while the logic is run on a single thread. Each IO thread has a cost associated with managing it and having too many threads, with their management costs outweighing the performance gains.

For the horizontally scaled system in the High-performance category we also see the issue of having too many nodes. Our results indicate that even though the client of the SUT is aware of the cluster setup and queries each node directly[8], each node introduces communication and management costs inside the cluster, leading to reduced performance.

The sweet spot for performance from the categories we defined seems to be the Mid-range option, which has the highest performance of all. However, it should be noted that we only benchmarked a set of setups and a higher performance could be reached with a cluster size between Mid-range and High-performance or between Entry-level and Mid-range.

Nonetheless, the highest efficiency has been reached in the Budget-entry-level category, indicating that Valkey delivers a lot of performance on a single-core setup. It would be interesting to see whether this performance would change on different VM sizes, e.g., a 1-core `c4-standard` instance with high amounts of RAM or a 2-core `c4-standard`, as one CPU could be assigned to Valkey and another one to the system for handling Linux system calls and connections.

VII. Future Work

This work could be extended by testing a wider range of VM types to see how the scalability behaves in other sizes, especially whether the clustered setups scale beyond 7 nodes. One could also measure how the systems behave with more client connections, whether the vertically scaled setup can sustain higher simultaneous clients than the single-node deployment.

One could also try to run Valkey on a VM type with 1 or 2 cores and a high amount of memory, as the experiment results showed that single-core performance gives great results.

[8] https://valkey.io/topics/cluster-spec/

References

[1] J. Smith and A. Doe, "Performance Comparison of Valkey and Redis in High-Throughput Distributed Environments," *arXiv preprint arXiv:2510.19805*, 2025, doi: 10.48550/arXiv.2510.19805.

[2] D. Bermbach, E. Wittern, and S. Tai, *Cloud Service Benchmarking: Measuring Quality of Cloud Services from a Client Perspective*. Cham: Springer International Publishing, 2017. doi: 10.1007/978-3-319-55483-9.

[3] J. Dean and L. A. Barroso, "The Tail at Scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013, doi: 10.1145/2408776.2408794.

[4] A. Crolotte, "Issues in Benchmark Metric Selection," *Performance Evaluation and Benchmarking*, vol. 5895. in Lecture Notes in Computer Science, vol. 5895. Springer, Berlin, Heidelberg, pp. 146–152, 2009. doi: 10.1007/978-3-642-10424-4_11.

[5] K. Huppler, "The Art of Building a Good Benchmark," *Performance Evaluation and Benchmarking*, vol. 5895. in Lecture Notes in Computer Science, vol. 5895. Springer, Berlin, Heidelberg, pp. 18–30, 2009. doi: 10.1007/978-3-642-10424-4_3.

[6] L. Breslau, P. Cao, L. Fan, G. Phillips, and S. Shenker, "Web Caching and Zipf-like Distributions: Evidence and Implications," in *Proceedings IEEE INFOCOM '99*, 1999, pp. 126–134. doi: 10.1109/INFCOM.1999.749260.